

Tutoriat 7

Stan Bianca-Mihaela, Stăncioiu Silviu

December 2020

CAND SILVIU DE LA ASC ARE NUMAI TAIIETURI,
TYPO-URI SI GRESELI IN MATERIALE CA SA
PREGATEASCA STUDENTII PENTRU CURSUL
STRUCTURI DE DATE DE PE SEMESTRUL 2



Contents

1 Rezolvarea subiectului 3, punctul c)

2

2 Recapitulare	6
2.1 Examen 2016	6
2.1.1 Subiectul I	6
2.1.2 Subiectul II	12
2.1.3 Subiectul III	17
3 Subiect 3 rezolvat la curs	24

1 Rezolvarea subiectului 3, punctul c)

Exemplul 1 [RENTANTA SEPTEMBRIE 2020]

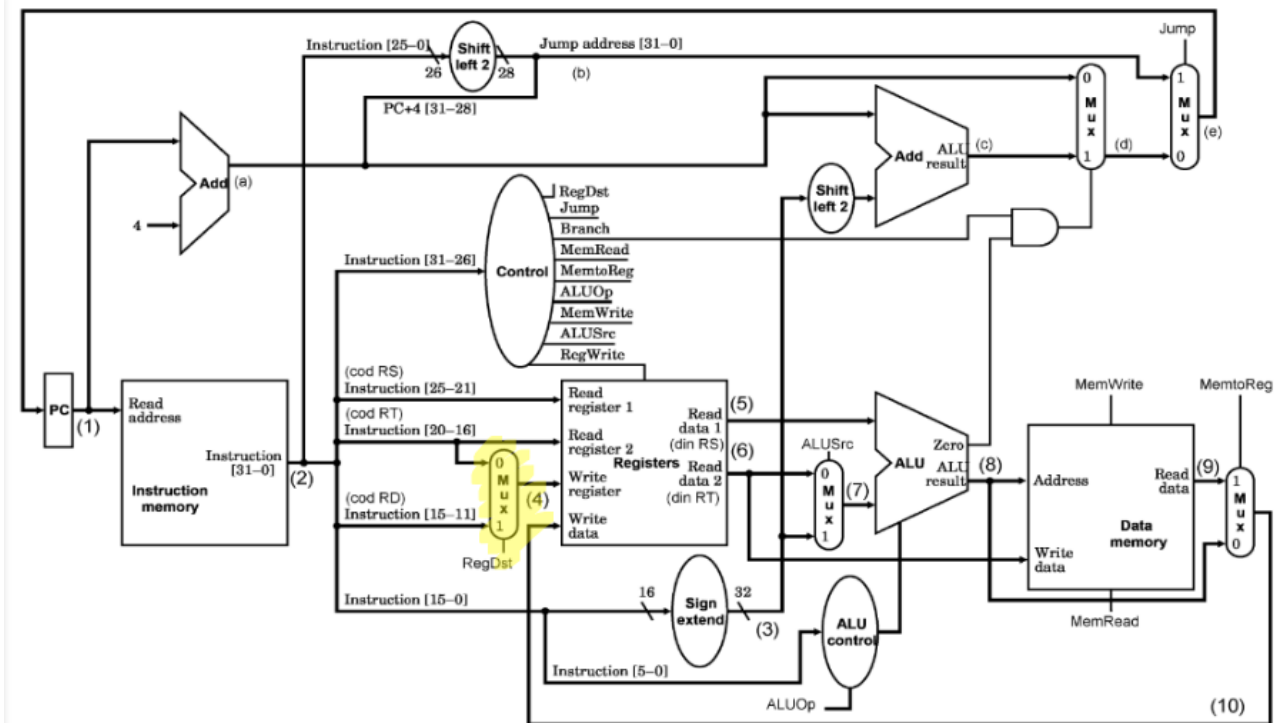
c) Adăugați procesorului implementarea instrucțiunii: `dlw rd, rs, rt` care încarcă în registrul `rd` valoarea aflată în memorie la adresa care se obține adunând valorile din regiștrii `rs` și `rt` (adică efectuează `rd := mem[rs + rt]`).

Pentru implementare, este suficientă adăugarea unei linii tabelului "Control" (codul `op = -` este o valoare nouă, nerelevantă). Completați această linie:

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—										

Sa analizam putin cerinta si sa luam coloanele pe rand:

- RegDst: Ce facea RegDst? Sa ne amintim din procesor:



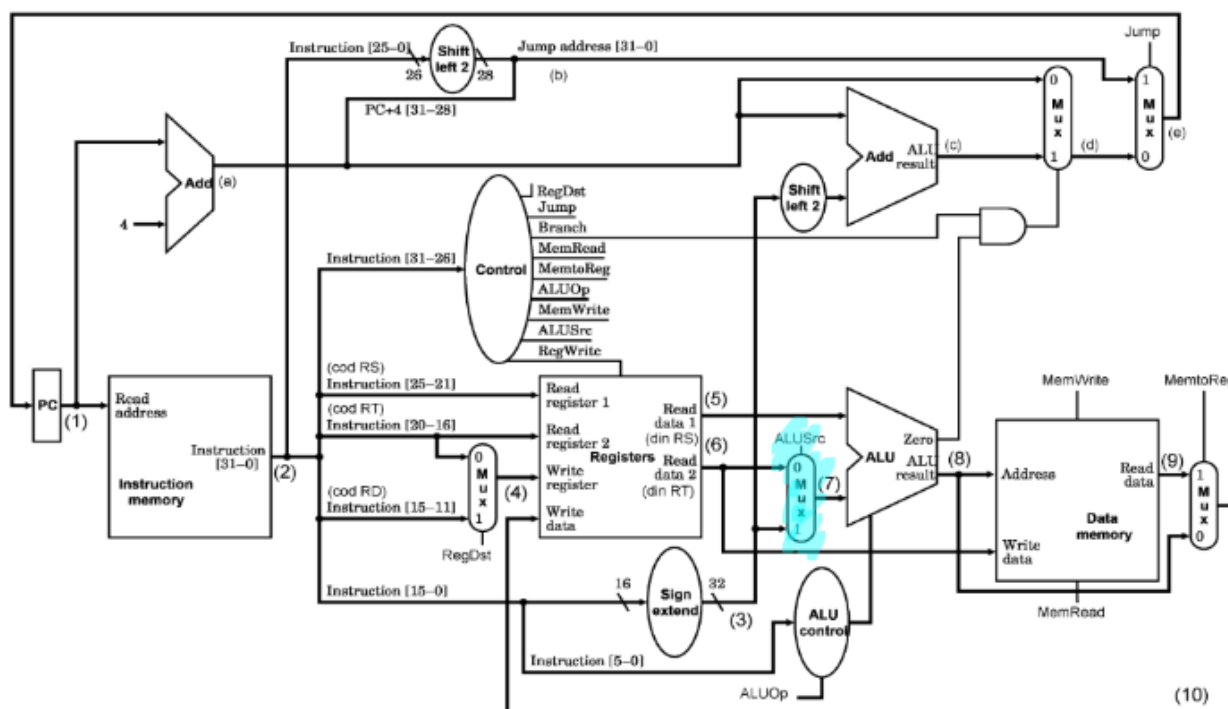
RegDst decide cine este registrul destinatie.

- Dacă avem o instrucțiune de tip R, registrul destinatie este `rd`. => `RegDst=1`
- Dacă avem o instrucțiune de tip I, cum ar fi `li $t0, 5`, registrul destinatie este chiar `rt`. => `RegDst=0`
- Când avem instrucțiuni de genul `sw` sau `beq`, nu avem niciun registru destinatie. => `RegDst=X` (aka. niciuna dintre variante).

Acum sa ne gandim ce fel de instructiune trebuie sa construim noi. Vedem din prima ca avem un rd, deci RegDst=1.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	<u>1</u>									

- ALUSrc: Ce facea ALUSrc?



Face diferenta dintre instructiunile care fac operatii pe valori si cele care fac operatii pe adrese de memorie.

- Pentru instructiunile de tip R si cele de tip branch o sa trimita ca input catre Unitatea Aritmetica si Logica rs si rt pentru ca ele fac operatii pe valori. (ALUSrc=0)
- Pentru instructiunile de tip I o sa trimita ca input catre Unitatea Aritmetica si Logica valoarea lui rs si valoarea imediata (imm) pentru ca vrem sa facem operatii pe niste adrese de memorie.(ALUSrc=1)

Din nou, noi avem o instructiune de tip R, deci ALUSrc va fi 0.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	<u>1</u>	<u>0</u>								

- MemToReg: Are voie instructiunea noastra sa citeasca din memorie si sa scrie intr-un registru? Da,

in cerinta scrie ca instructiunea **incarca in rd** o valoare aflata la o **adresa de memorie**. => Mem-ToReg=1.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1							

- RegWrite: Are voie instructiunea noastra sa scrie intr-un registru? Clar. => RegWrite=1.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1						

- MemRead: Are voie instructiunea noastra sa citeasca din memorie? Da. => MemRead=1.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1	1					

- MemWrite: Are voie instructiunea noastra sa scrie din memorie? Nu. => MemWrite=0.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1	1	0				

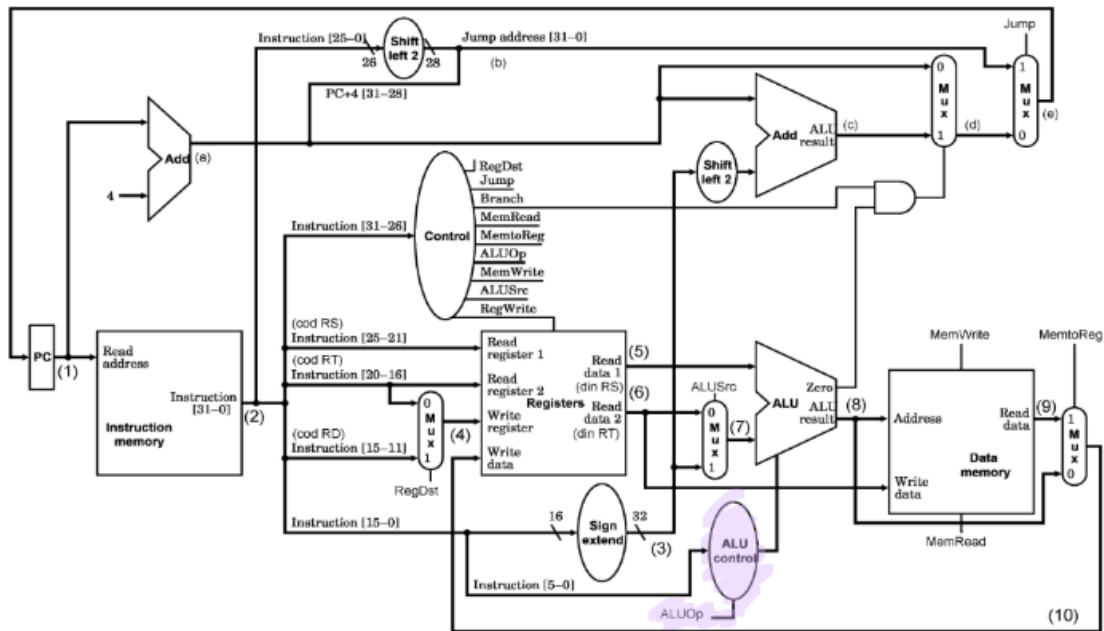
- Branch: E instructiunea noastra de tip branch? Nu. => Branch=0.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1	1	0	0			

- Jump: E instructiunea noastra de tip branch? Nu. => Jump=0.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1	1	0	0	0		

- ALUOp (o sa tratam si ALUOp1 si ALUOp2 intr-o singura sectiune pentru ca e aceeaasi discutie pe ambele).
Vom face o analiza mai amanuntita pentru ALUOp. Stim ca ALUOp intra in ALUcontrol, care scoate ce fel de operatie va efectua Unitatea Aritmetica si Logica (ALU).



Dar ce mai exact inseamna aceste perechi de numere (ALUOp1, ALUOp2)?

- (ALUOp1=0, ALUOp2=0) : fa intotdeauna adunare (addi, lw, sw, etc.)
- (ALUOp1=0, ALUOp2=1) : fa intotdeauna scadere (beq, bne, etc.)
- (ALUOp1=1, ALUOp2=X) : fa o operatie determinata de campul Funct (din instructiunile de tip R)
exemplu: La add vrem sa facem adunare, la sub scadere, etc.

Acum ca stim asta, putem sa alegem valorile pe care le vrem noi. Instructiunea noastra trebuie sa:

- determine adresa de memorie de la care sa citeasca, prin adunare
- sa incarce in registrul rd valoarea gasita la acea adresa de memorie, tot prin adunare (daca ne uitam la valorile lui ALUOp1 si ALUOp2 din tabelele de ajutor avem perechea (0,0), care inseamna adunare).

Deci vom alege adunarea, adica ALUOp1=0, ALUOp2=0.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—	1	0	1	1	1	0	0	0	0	0

2 Recapitulare

Ne apucăm să rezolvăm subiecte date în alți ani la examenul de ASC. Vom începe cu subiectele date în 2016.

2.1 Examen 2016

2.1.1 Subiectul I

Fie $x = 73.75$ și $y = 5.25$

- Converțiți x și y în baza 2.
- Converțiți mai departe x și y din baza 2 în baza 16, direct (fără a trece prin baza 10).
- Calculați $x - y$ lucrând în baza 16.
- Converțiți rezultatul din baza 16 în baza 10.
- Calculați $73.75 + (-5.25)$ folosind algoritmul de adunare în virgulă mobilă pentru formatul single (se va lucra cu reprezentările matematice în baza 2 în notație științifică, iar în final se va converti rezultatul în baza 10).
- Determinați reprezentarea internă ca single a lui x , binară (32 biți) și hexa (8 cifre hexa).
- Interpretați ca număr în baza 10 reprezentarea internă hexa în virgulă mobilă 0x8C, considerând formatul dat de $n = 8$ (dimensiunea locației) și $k = 2$ (dimensiunea câmpului caracteristică).

Rezolvare:

a)

$$(73,75)_2 = ?$$

$$\begin{array}{r|l} 73 & 1 \\ 36 & 0 \\ 18 & 0 \\ 9 & 1 \\ 4 & 0 \\ 2 & 0 \\ 1 & 1 \end{array} \quad \Rightarrow (73)_2 = \overline{1001001}$$

$$\begin{array}{l} 0,75 \cdot 2 = 1,5 \\ 0,5 \cdot 2 = 1,0 \end{array} \quad \Rightarrow (0,75)_2 = \overline{0,11}$$

$$(5,25)_2 = ?$$

$$\begin{array}{r|l} 5 & 1 \\ 2 & 0 \\ 1 & 1 \end{array} \quad \Rightarrow (5)_2 = \overline{101}$$

$$\begin{array}{l} 0,25 \cdot 2 = 0,5 \\ 0,5 \cdot 2 = 1,0 \end{array} \quad \Rightarrow (0,25)_2 = \overline{0,01}$$

$$\Rightarrow (5,25)_2 = \overline{101,01}$$

b)

$16 = 2^4 \Rightarrow$ grupăm câte 4

01001001, 1100

$$(\overline{0100})_2 = (4)_{16}$$

$$(\overline{1001})_2 = (9)_{16}$$

$$(\overline{1100})_2 = (C)_{16}$$

$$\Rightarrow x = (\overline{49,C})_{16}$$

0101, 0100

$$(\overline{0101})_2 = (5)_{16}$$

$$(\overline{0100})_2 = (4)_{16}$$

$$\Rightarrow y = (\overline{5,4})_{16}$$

c) Vom înmulți cele două numere cu 16 pentru a scăpa de virgulă, iar rezultatul îl vom înmulți cu 16 pentru a obține răspunsul corect.

$$\begin{array}{r} 49C - \\ 54 \\ \hline 448 \end{array}$$

$$\Rightarrow x - y = (\overline{44,8})_{16}$$

d)

$$(44,8)_{16}^{-1} = ?$$

$$(44,8)_{16}^{-1} = 4 \cdot 16^1 + 4 \cdot 16^0 + 8 \cdot 16^{-1} =$$
$$= 64 + 4 + \frac{8}{16} = 68,5$$

e)

$$(\overline{1001001,11})_2 = (\overline{1,00100111}) \cdot 2^6$$

$$(\overline{-101,01})_2 = (\overline{-1,0101})_2 \cdot 2^2$$

OVERFLOW / UNDERFLOW :

$$-126 \leq G \leq 127 \quad \checkmark$$

$$-126 \leq Q \leq 127 \quad \checkmark$$

ROUTINE

$$|p_x| = |00100111| = 6 \leq 23$$

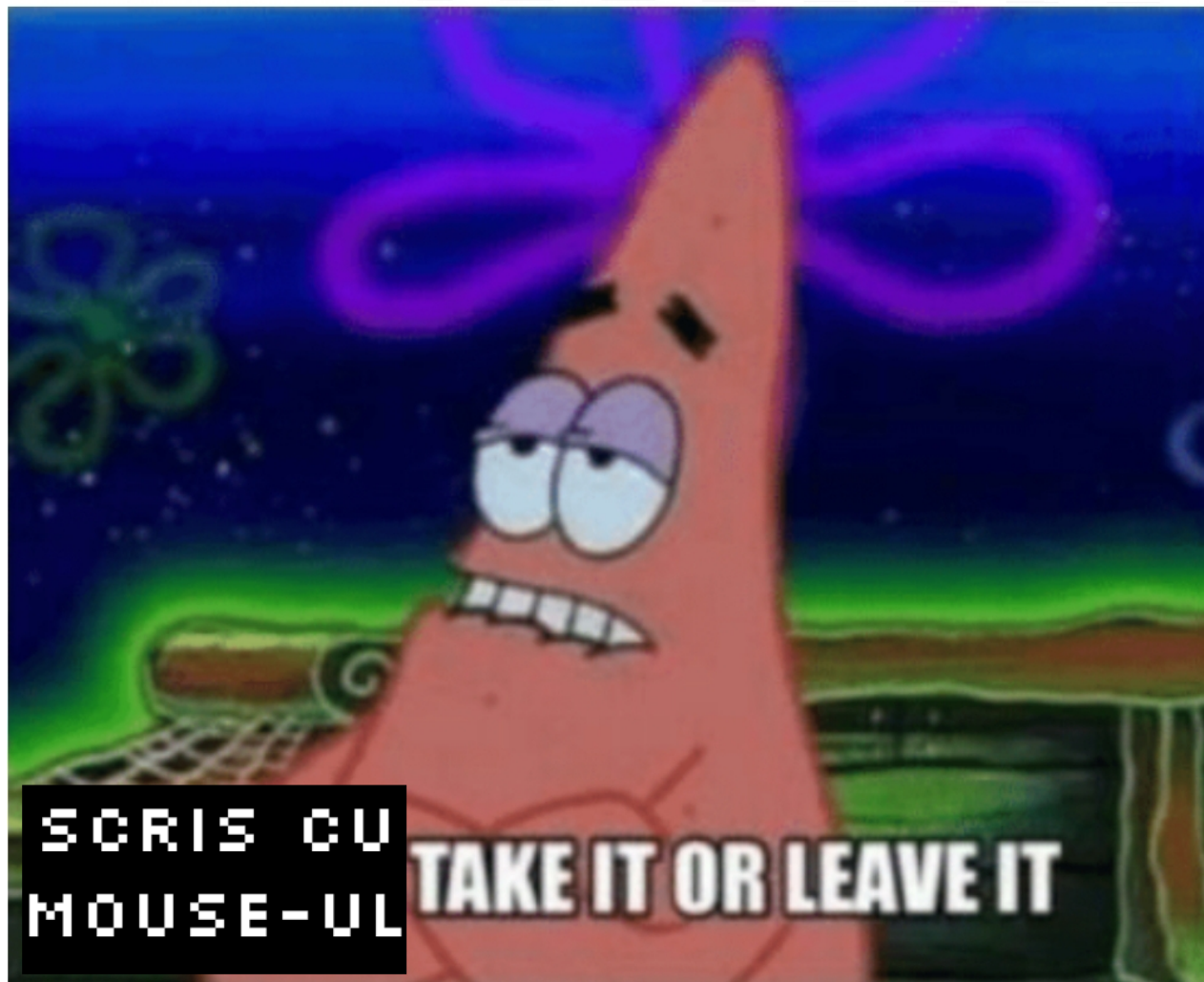
$$|p_y| = |0101| = 4 \leq 23$$

$2 < 6$ $\Rightarrow$ add exp Q to x

$$\Rightarrow y = -0,00010101 \cdot 2^6$$

$$\begin{array}{r} 1,00100111 - \\ 0,00010101 \\ \hline 1,00010010 \end{array}$$

CAND NU AI TABLETA
GRAFICA SI TI-E LENE SA
SCRII TUTORIATUL IN LATEX



f)

Reprezentare internă

$$m=32, k=8, p=24$$

$$x = (1,00100111)_2 \cdot 2^6$$

$$2) E = 6$$

$$p = 00100111 - \text{mantina}$$

$$s = 0 \text{ (positiv)}$$

$$c = E + \text{BIAS} = 6 + 127 = 133$$

TESTE

OVERFLOW / UNDERFLOW

$$-126 \leq E \leq 127 \quad \checkmark$$

ROUNDSIRE

$$|p| = 6 \leq 23 \quad \checkmark$$

$$2) x: \underbrace{0}_s \underbrace{10000101}_c \underbrace{0010011100 \dots 0}_{23-8=15 \text{ - wu}}$$

mantina

$$\text{HEXA} : 42938000$$

g)

INTERPRETARE 0x8C

$$m = 8 \quad k = 2$$

$$\begin{array}{l} b = \overline{1000} \\ c = \overline{1100} \end{array} \quad \left| \begin{array}{l} \rightarrow \text{reprezentarea binară:} \\ 10001100 \end{array} \right.$$

s: 1 (primul bit, nu știm dacă avem nr negativ)
 c: 00 (următoarele $k = 2$ cifre)
 f: 01100 (mantina)

c = 00 $\rightarrow$ format dinormalizat

$$\rightarrow x = (-1)^s \cdot 2^{E_{min}} \cdot 0, f$$

$$E_{min} = -(2^{k-L} - 2) = 0$$

$$x = -0, f = -0,01100$$

$$(x)_2^{-1} = -0,375$$

2.1.2 Subiectul II

Fie $f : B_2^3 \rightarrow B_2^3, f(x, y, z) = (f_1(x, y, z), f_2(x, y, z))$, unde $f_1, f_2 : B_2^3 \rightarrow B_2, f_1(x, y, z) = y + \bar{x}z, f_2(x, y, z) = 1$ dacă și numai dacă cel puțin două dintre variabilele x, y, z au valoarea 1.

a) Construiți tabelul de valori al lui f și scrieți f_1, f_2 în FND și FNC.

b) Implementați f folosind un PROM.

c) Implementați f folosind un codificator.

d) Implementați f folosind multiplexori elementari, apoi reduceți la maximum numărul de multiplexori elementari, iar în final redesați circuitul păstrând doar multiplexorii rămași.

e) Desenați un circuit 1-DS care, primind la intrare un șir de biți, scoare la ieșire 1 dacă și numai dacă cel puțin doi dintre ultimii 3 biți au valoarea 1.

f) Considerăm automatul $(Q, X, Y, \delta, \lambda)$, unde:

$$Q = \{0, 1\}^2, X = Y = \{0, 1\},$$

$$\delta(q_1, q_0, x) = (f_1(q_1, q_0, x), f_2(q_1, q_0, x)), \lambda(q_1, q_0, x) = f_1(q_1, q_0, x) \oplus f_2(q_1, q_0, x).$$

Implementați acest automat în maniera standard, folosind PLA și TFF.

Rezolvare:

a)

Tabelul pentru aceasta functie este:

index	x	y	z	$\bar{x}$	$\bar{z}$	$f_1(x, y, z)$	$f_2(x, y, z)$
(0)	0	0	0	1	0	0	0
(1)	0	0	1	1	1	1	0
(2)	0	1	0	1	0	1	0
(3)	0	1	1	1	1	1	1
(4)	1	0	0	0	0	0	0
(5)	1	0	1	0	0	0	1
(6)	1	1	0	0	0	1	1
(7)	1	1	1	0	0	1	1

FMD

$$f_1 = \underbrace{\bar{x} \cdot \bar{y} \cdot z}_{(2)} + \underbrace{\bar{x} \cdot y \cdot \bar{z}}_{(3)} + \underbrace{\bar{x} \cdot y \cdot z}_{(4)} + \underbrace{x \cdot y \cdot \bar{z}}_{(6)} + \underbrace{x \cdot y \cdot z}_{(7)}$$

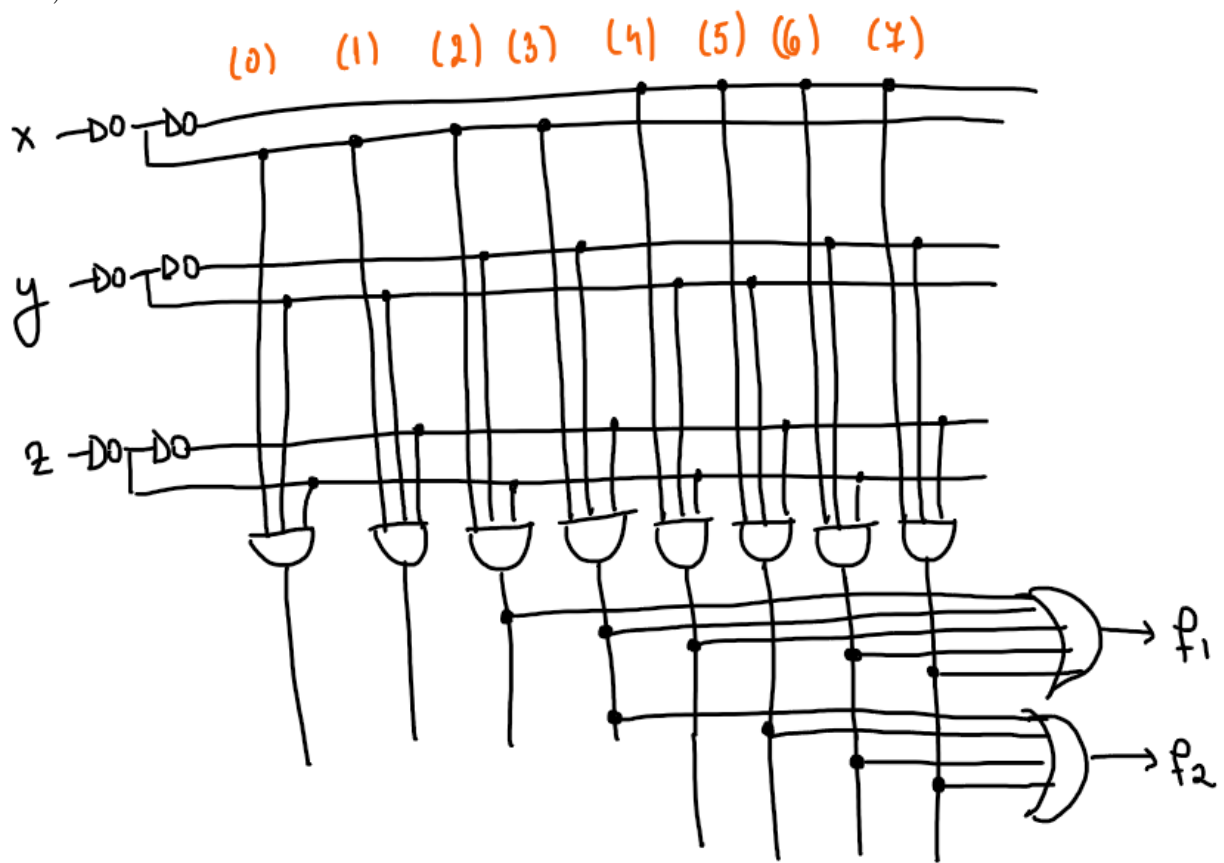
$$f_2 = \underbrace{\bar{x} \cdot y \cdot z}_{(3)} + \underbrace{x \cdot \bar{y} \cdot z}_{(5)} + \underbrace{x \cdot y \cdot \bar{z}}_{(6)} + \underbrace{x \cdot y \cdot z}_{(7)}$$

FMC

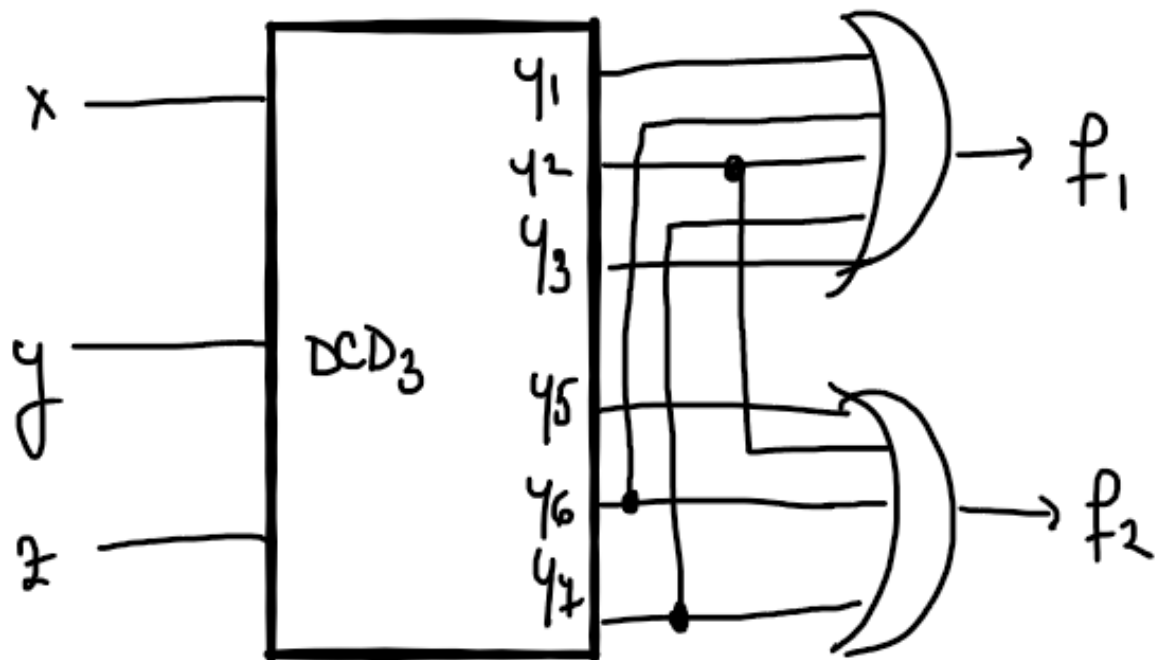
$$f_1 = (\underbrace{x + y + z}_{(0)}) \cdot (\underbrace{\bar{x} + y + z}_{(4)}) \cdot (\underbrace{\bar{x} + y + \bar{z}}_{(5)})$$

$$f_2 = (\underbrace{x + y + z}_{(0)}) \cdot (\underbrace{x + y + \bar{z}}_{(1)}) \cdot (\underbrace{x + \bar{y} + z}_{(2)}) \cdot (\underbrace{\bar{x} + y + z}_{(4)})$$

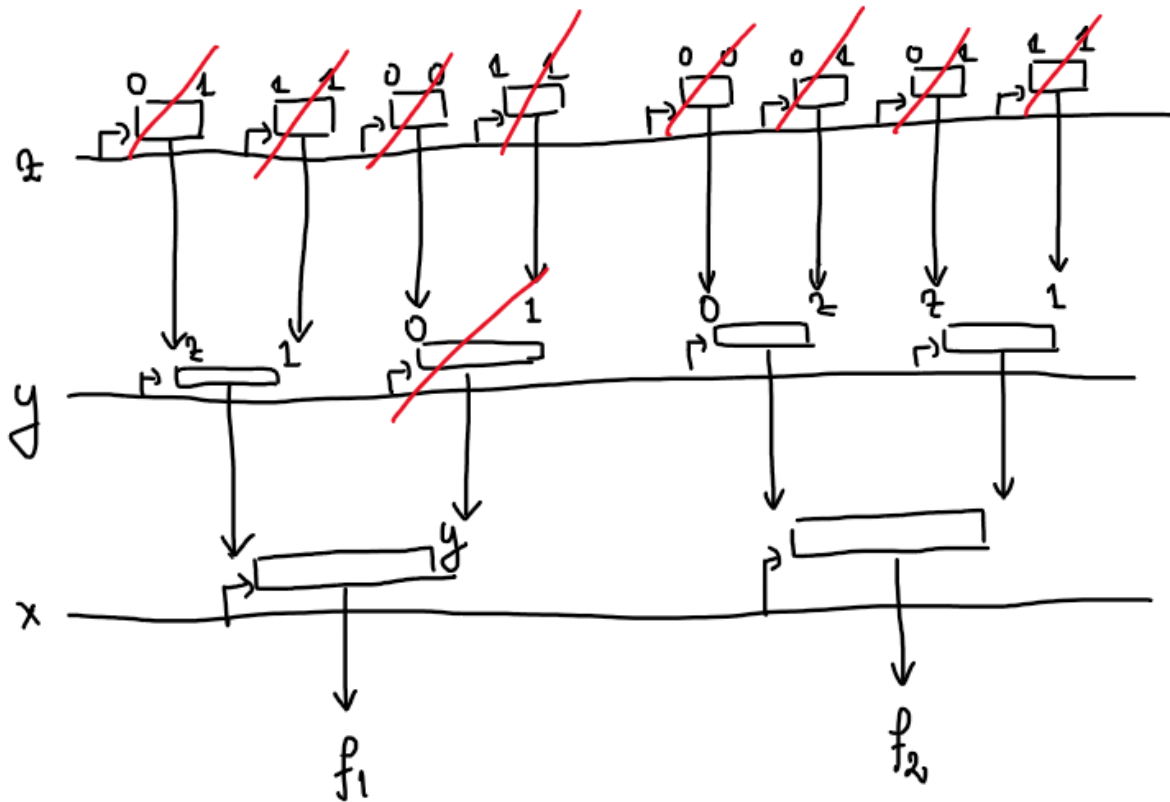
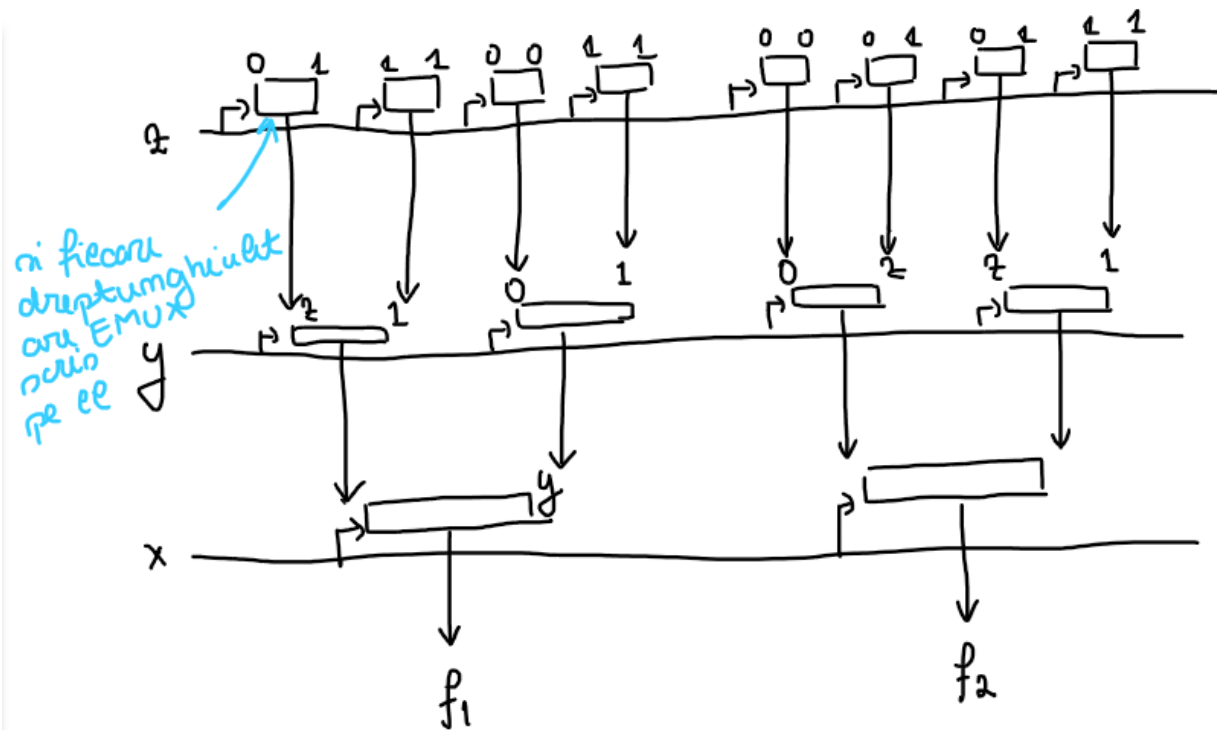
b)



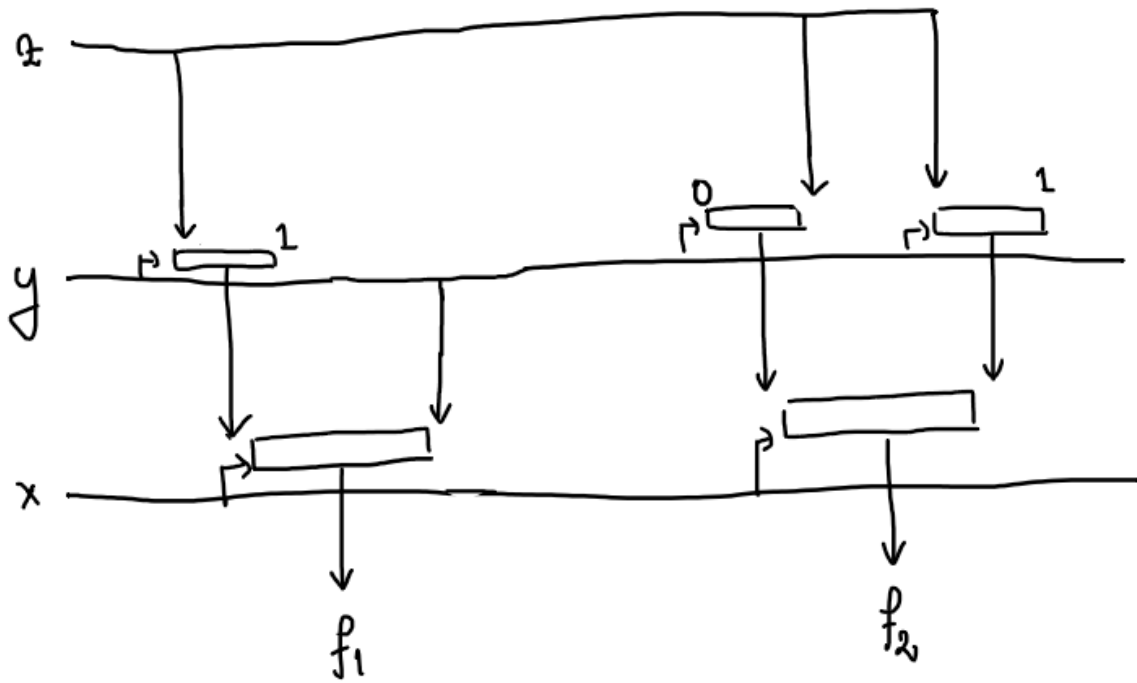
c)



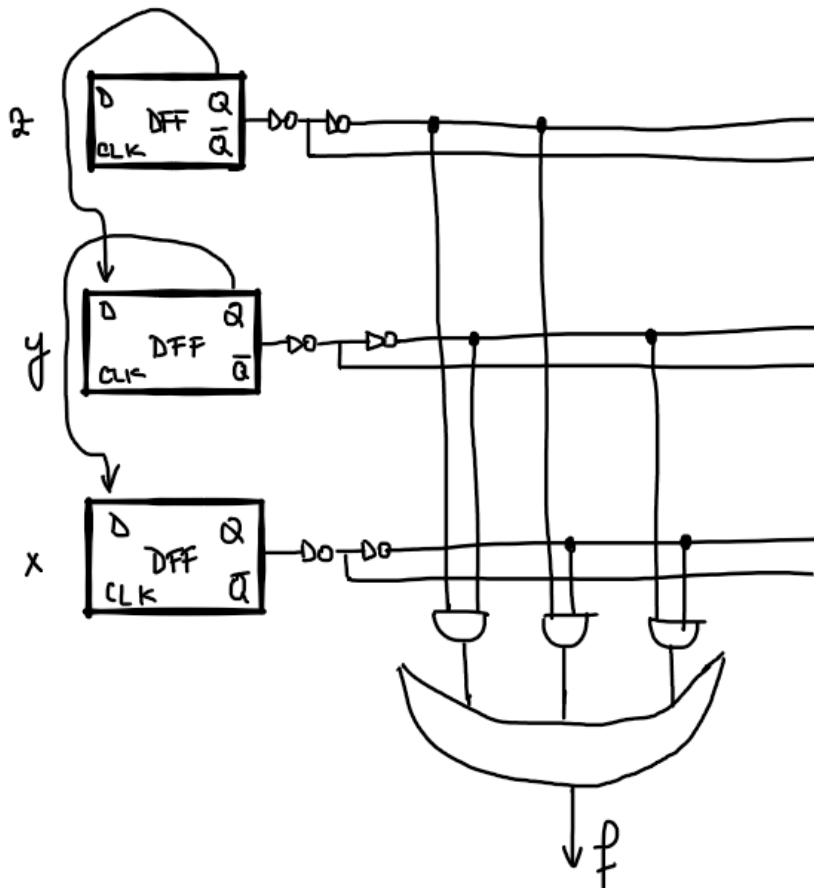
d)



Minimizarea multiplexorilor elementari:



e)



f) Punctul asta are automate, pe care o sa le faceti la Limbaje Formale si Automate. Anul trecut a zis ca nu o sa dea asa ceva, cel mai probabil asa va fi si la voi.

2.1.3 Subiectul III

Consideram implementarea procesorului MIPS cu 1 ciclu/ instructiune. Fie programul:

```
.data
x: .word 3
y: .word 4
.text
main:
la $t1, x
li $t2, 2
lw $t3, y
et:
sub $t3, $t3, $t2
sw $t3, 4($t1)
beq $t3, $t2, et
li $v0, 10
syscall
```

Presupunem ca in memorie instructiunea sw din program are adresa α , iar variabila x are adresa $\beta=0x10010000$.

a) Pentru instructiunile sw si beq din program scrieti campurile din reprezentarea lor interna (ex: op/rs/rt/imm, valorile se scriu in hexa); pentru beq din program scrieti reprezentarile ei binara (32 biti) si hexa (8 cifre hexa).

Incepem cu sw \$t3, 4(\$t1). Ne uitam in pdf-ul cu instructiunile MIPS (gasiti un link la el si in tutoriatul 6 si in Teams, la Files). Vedem ca sw are forma sw \$t, offset(\$s) si encoding-ul:

1010 11ss ssst tttt iiii iiii iiii

In tutoriatul 6 avem o poza cu scris de mana care ne spune ce cod are fiecare registru. Deci:

- $\$t3 = (11)_6 = \$t = \overline{B}$ (in hexa)
- $\$t4 = (12)_6 = \$s = \overline{C}$ (in hexa)
- $\text{offset} = (4)_6 = \overline{4}$ (in hexa)

Care e valoarea lui opcode in hexa?

$$\overline{(101011)}_2 = \overline{(2B)}_6$$

Deci reprezentarea in memorie a lui sw putem sa o scriem ca:

sw: 2B/C/B/4 (reprezentarea asta cu / respecta formatul din cerinta)

Cea pe biti este:

sw: 1010 1101 1000 1011 0000 0000 0000 0100 (in binar)

sw: AD8B0004 (in hexa).

Pentru beq (beq \$t3, \$t2, et) stim ca are forma beq \$s, \$t, offset si encoding-ul:

0001 00ss ssst tttt iiii iiii iiii

Din nou, care sunt registri:

- $\$t3 = (10)_2 = \$t = \overline{01010}$ (pe 5 biti)
- $\$t2 = (11)_6 = \$s = \overline{01011}$ (pe 5 biti)

- $\text{offset} = (-3)_{16}$
Cum scriem -3 pe 16 biti? Complement fata de 2.
Reprezentarea lui 3 pe 16 biti: 0000 0000 0000 0011. Complementul fata de 1 si apoi adun 1.

$$\begin{array}{r}
 \boxed{2} \quad 0000 \quad 0000 \quad 0000 \quad 0011 \\
 \hline
 \quad 1111 \quad 1111 \quad 1111 \quad 1100 \quad + \\
 \quad \quad \quad \quad \quad \quad \quad 1 \\
 \hline
 \quad 1111 \quad 1111 \quad 1111 \quad 1101
 \end{array}$$

Deci $\text{offset} = \overline{1111111111111100}$.

Aici ne cere explicit reprezentarea pe biti (deci nu cea cu /). Aceasta va fi:

beq: 0001 0001 0100 1011 1111 1111 1111 1100 (in binar)

beq: 114BFFFC (in hexa)

b) Completati tabelul urmatoar cu valorile obtinute la prima executare a instructiunilor sw si beq din program (am notat prescurtat: B=Branch, MR=MemRead, MW=MemWrite, iar Mem[y] inseamna continutul variabilei y); valorile se scriu hexa/formula, iar daca valoarea este necunoscuta/nedefinita o vom nota cu "?"; in coloanele PC si Mem[y] se vor trece valorile noi, de la sfarsitul fiecarei instructiuni executate:

Inainte sa trecem mai departe trebuie sa analizam ce se intampla la fiecare instructiune:

- Ce se intampla pana la sw?
x are valoarea 3
y are valoarea 4
\$t1 tine adresa lui x, adica β
\$t2 are valoarea 2
\$in y pun valoarea lui \$t3, adica 2
din \$t3 se scade \$t2 si de pune la loc in \$t3, deci \$t3 are acum valoarea 2
- Ce se intampla pana la beq?
la sw am pus in y valoarea din \$t3, care este 2

Incipem cu valorile care pot fi luate din tabelele de ajutor:

	1	3	5	7	8	ALU zero	(d)	(e)	b	M2	MW	ALU Src	ALU Op (2b)	ALU Ctrl (3b)	PC	Mem[4]
Initial	—	—	—	—	—	—	—	—	—	—	—	—	—	—	α	5
sw	α								0	0	1	1	00	010		
beq									1	0	0	0	x1	110		

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch
R-format	1	0	0	1	0	0	0
lw	0	1	1	1	1	0	0
sw	X	1	X	0	0	1	0
beq	X	0	X	0	0	0	1

na ignorați complet
questiunile 2
ce e oare

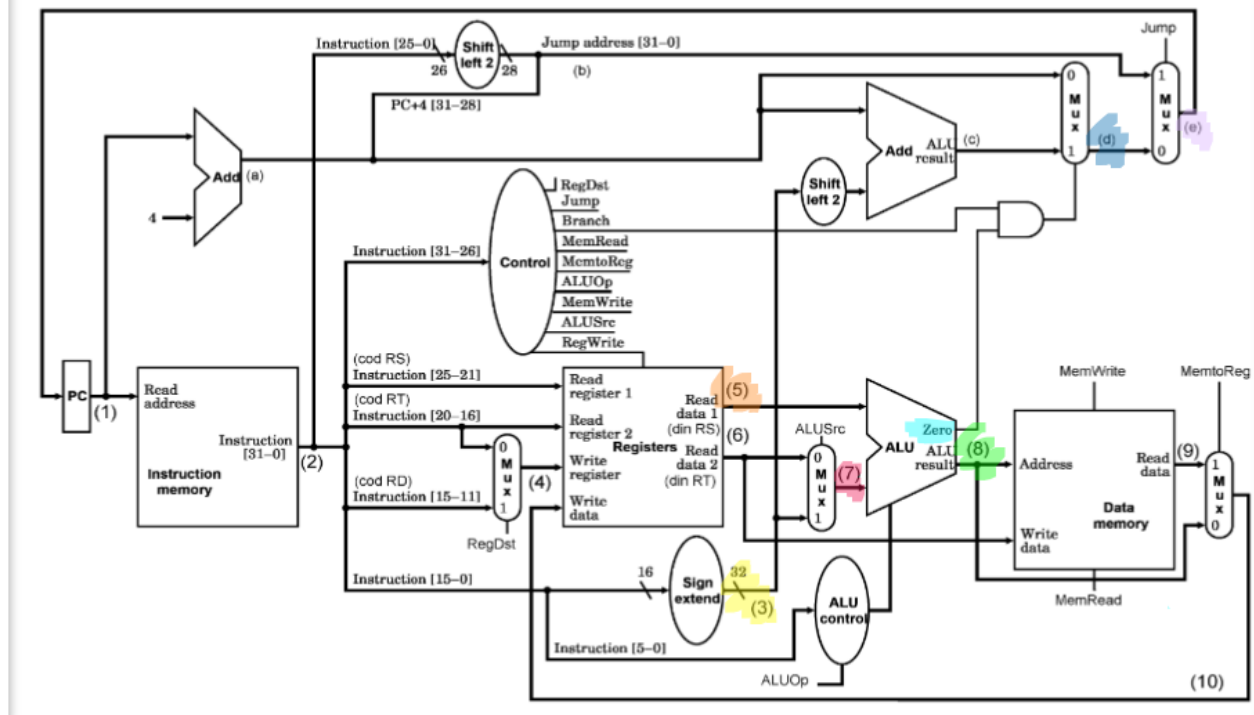
ALU Control (slide 7.36)

		ALUOp		Camp functie						Operatie
		ALUOp ₁	ALUOp ₀	F5	F4	F3	F2	F1	F0	
lw/sw beq add sub and or R-format slt	0	0	X	X	X	X	X	X	010	(+)
	X	1	X	X	X	X	X	X	110	(-)
	1	X	X	X	0	0	0	0	010	(+)
	1	X	X	X	0	0	1	0	110	(-)
	1	X	X	X	0	1	0	0	000	(and)
	1	X	X	X	0	1	0	1	001	(or)
	1	X	X	X	1	0	1	0	111	(slt)

Continuam cu celelalte valori. Le-am completat cu valorile deja in hexa. Sunt fix chestiile pe care le-am facut tutoriatul trecut, deci exercitiul asta ar trebui sa fie o verificare pentru voi.

	1	3	5	7	8	ALU zero	(d)	(e)	b	MZ	MW	ALU Src	ALU Op (2b)	ALU Ctrl (3b)	PC	Mem[4]
Initial	—	—	—	—	—	—	—	—	—	—	—	—	—	—	α	5
SW	α	4	C	4	10	0	$\alpha + \alpha$	$\alpha + \alpha$	0	0	1	1	00	010	$\alpha +$	2
beq	$\alpha + 8$	-3	2	2	0	1	α	α	1	0	0	0	X1	110	α	2

$\uparrow (\alpha + 12) + (-3) \cdot 4 = \alpha$

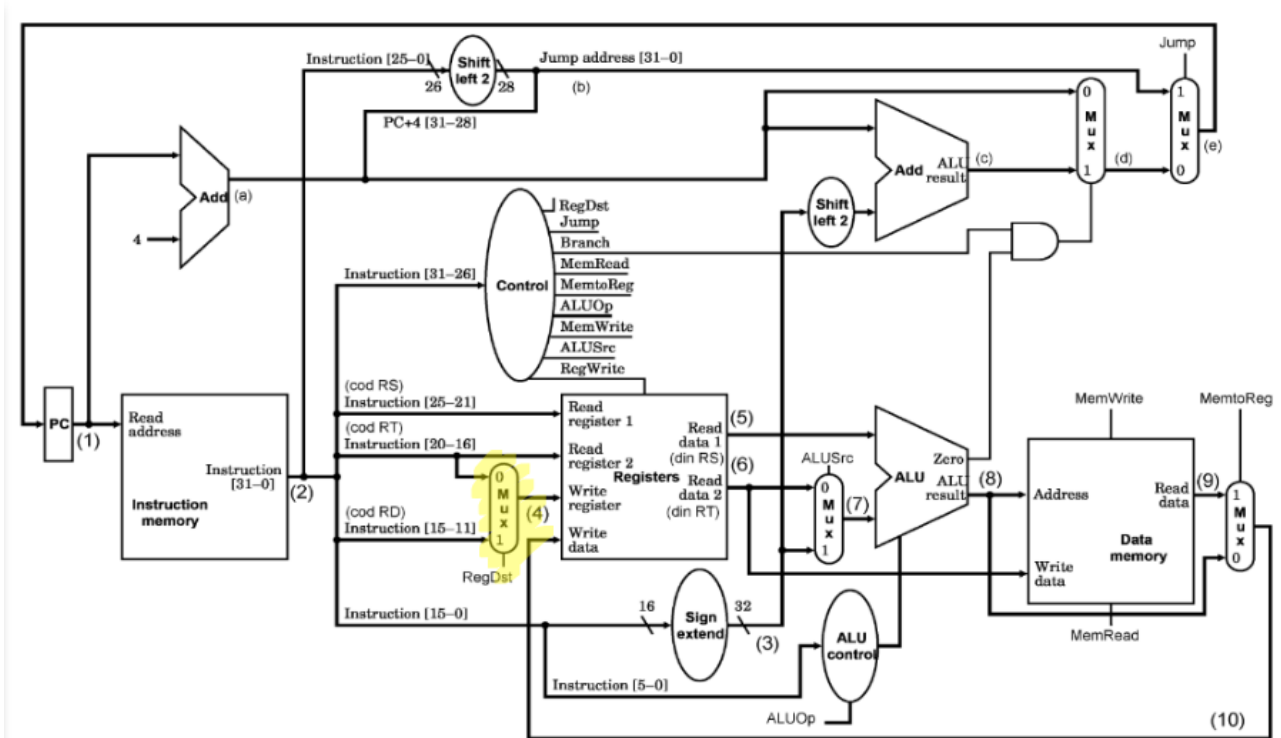


c) Adagati procesorului implementarea **subp rt, rs** care atribuie registrului rt diferenta valorilor din registrii **rs** si **rt** (adica efectueaza $rt := rs - rt$); se va folosi formatul I, cu $op = 0xFF$ si $imm = 0x0$. Pentru implementare, este suficienta adaugarea unei linii in taelul "Control". Completati aceasta linie:

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
—										

Incepem sa analizam:

- RegDst: Ce facea RegDst? Sa ne amintim din procesor:

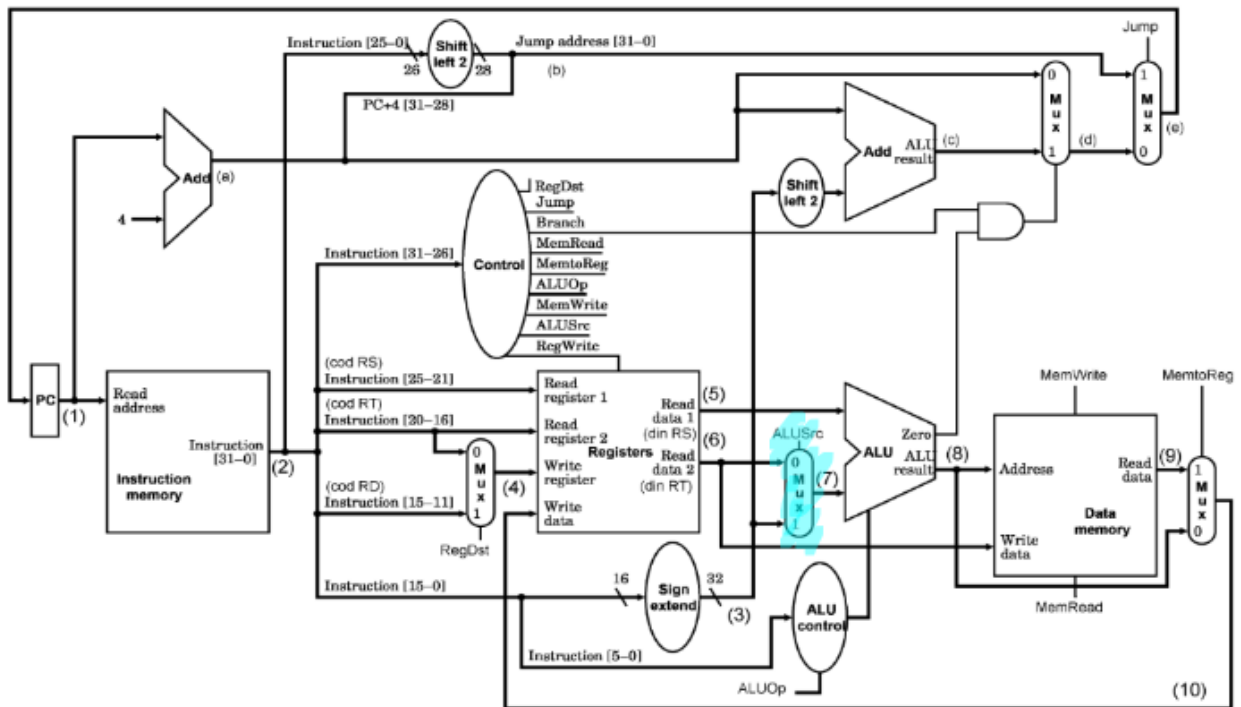


RegDst decide cine este registrul destinatie.

- Daca avem o instructiune de tip R, registrul destinatie este rd. => RegDst=1
- Daca avem o instructiune de tip I, cum ar fi li \$t0, 5, registrul destinatie este chiar rt. => RegDst=0
- Cand avem instructiuni de genul sw sau beq, nu avem niciun registru destinatie. => RegDst=X (aka. niciuna dintre variante).

Acum sa ne gandim ce fel de instructiune trebuie sa construim noi. Vedem din prima ca registrul destinatie este rt. => RegDst=0

- ALUSrc: Ce facea ALUSrc?



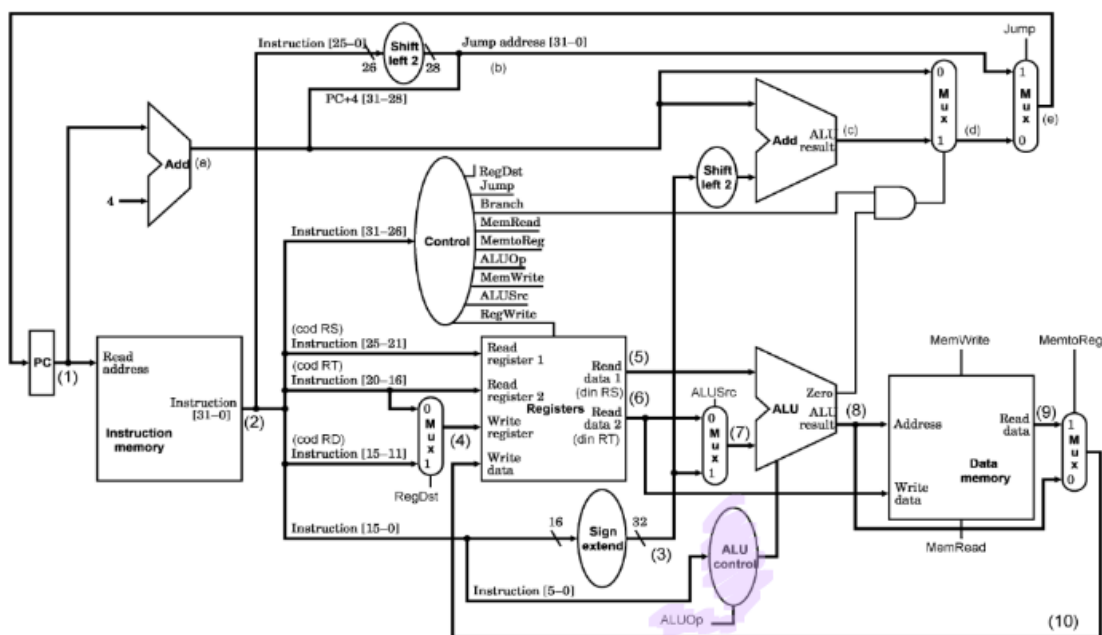
Face diferenta dintre instructiunile care fac operatii pe valori si cele care fac operatii pe adrese de memorie.

- Pentru instructiunile de tip R si cele de tip branch o sa trimita ca input catre Unitatea Aritmetica si Logica rs si rt pentru ca ele fac operatii pe valori. (ALUSrc=0)
- Pentru instructiunile de tip I o sa trimita ca input catre Unitatea Aritmetica si Logica valoarea lui rs si valoarea imediata (imm) pentru ca vrem sa facem operatii pe niste adrese de memorie.(ALUSrc=1)

Noi vrem sa facem diferenta dintre 2 valori, nu lucram cu adrese de memorie, deci alegem ALUSrc=0.

- MemToReg: Are voie instructiunea noastra sa citeasca din memorie si sa scrie intr-un registru? Nu, pentru ca nu are voie sa citeasca din memorie. MemToReg=0.
- RegWrite: Are voie instructiunea noastra sa scrie intr-un registru? Da. RegWrite=1.
- MemRead: Are voie instructiunea noastra sa citeasca din memorie? Nu. => MemRead=0.
- MemWrite: Are voie instructiunea noastra sa scrie din memorie? Nu. => MemWrite=0.
- Branch: E instructiunea noastra de tip branch? Nu. => Branch=0.
- Jump: E instructiunea noastra de tip branch? Nu. => Jump=0.
- ALUOp (o sa tratam si ALUOp1 si ALUOp2 intr-o singura sectiune pentru ca e aceeaasi discutie pe ambele).

Vom face o analiza mai amanuntita pentru ALUOp. Stim ca ALUOp intra in ALUcontrol, care scoate ce fel de operatie va efectua Unitatea Aritmetica si Logica (ALU).



Dar ce mai exact inseamna aceste perechi de numere (ALUOp1, ALUOp2)?

- (ALUOp1=0, ALUOp2=0) : fa intotdeauna adunare (addi, lw, sw, etc.)
- (ALUOp1=0, ALUOp2=1) : fa intotdeauna scadere (beq, bne, etc.)
- (ALUOp1=1, ALUOp2=X) : fa o operatie determinata de campul Funct (din instructiunile de tip R)
exemplu: La add vrem sa facem adunare, la sub scadere, etc.

Acum ca stim asta, putem sa alegem valorile pe care le vrem noi. Instructiunea noastra trebuie sa:

- sa faca diferenta rs - rt, deci in Unitatea Aritmetica si Logica avem nevoie ori de ALUOp1=0, ALUOp2=1 (scadere intotdeauna), ori de ALUOp1=1, ALUOp2=X (fa ce operatie iti indica campul funct din reprezentarea in memorie a instructiunii). De ce nu putem alege ALUOp1=1, ALUOp2=X? Pentru ca instructiunea noastra **subp rt, rs** nu este de tip R => nu are in reprezentarea sa in memorie campul funct.
- sa puna rezultatul in rt (aici nu ne trebuie Unitatea Aritmetica si Logica, daca ne uitam in procesor, rezultatul scaderii iese pe la (8), datorita lui MemToReg se transmite catre (10) si de acolo se duce direct in Write Data, adica valoarea care va fi incarcata in registrul destinatie).

=> Aleg ALUOp1=0, ALUOp2=1.

Instruction [31 - 26]	RegDst	ALU Src	Mem To Reg	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU Op1	ALU Op0
0xFF	0	0	0	1	0	0	0	0	0	1

3 Subiect 3 rezolvat la curs

La noi cel puțin, al ultimul curs a rezolvat Draguliciun subiect 3, pe care l-a completat destul de interesant așa ca să îl facem și noi.

Bucata de cod utilizata pentru a completa tabelul este:

```
# x=2*y
.data
x: .space 4
y: .word 10
.text
main:
la $t0,x
lw $t1,4($t0)
add $t2,$t1,$t1
sw $t2,0($t0)
li $v0,10
syscall
```

Tabelul completat este:

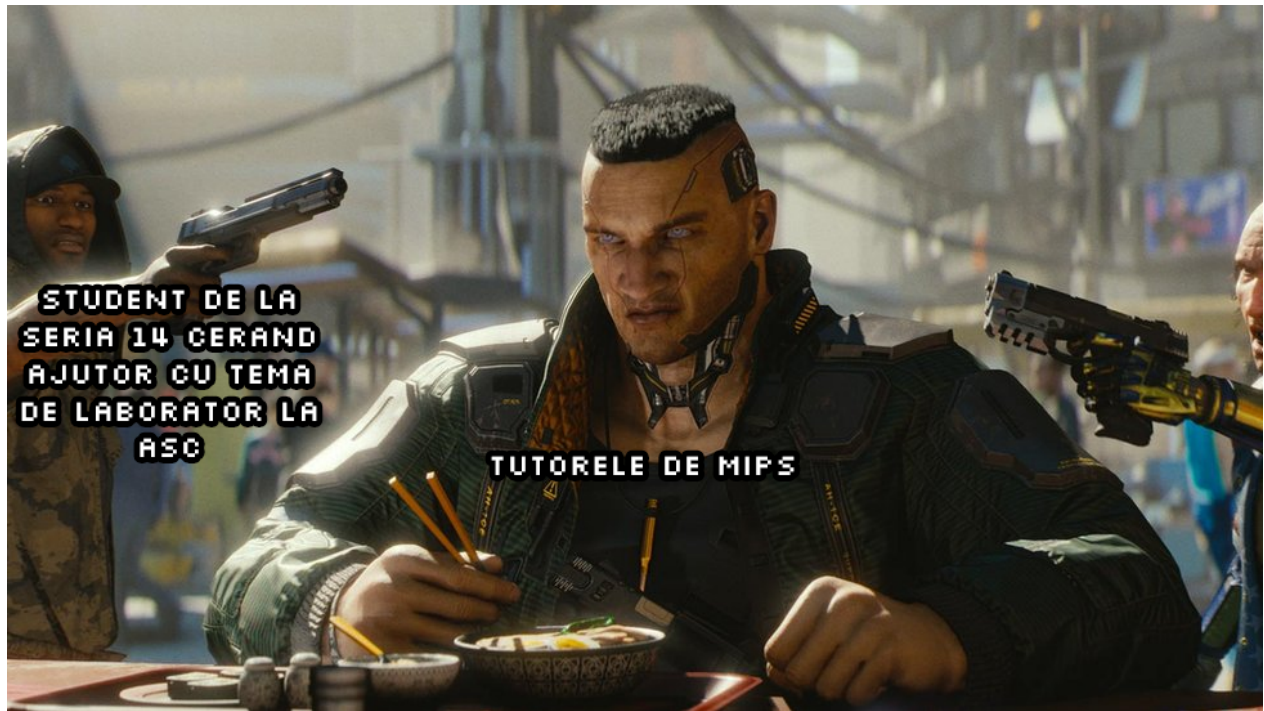
	op/rs/rt/rd/o/fct	(s)	CTRL RegDst/ALUSrc/MemToReg RegWrite/MemRead/ MemWrite/Branch/ Jump/ALUOp	ALU ctrl	(4)	(5)	(6)	(7)	ALU result (8)	ALU zero	(9)	(10)	(e)
lw	α 23/8/9/4	4	0111000/00	010 +	9	p	?	4	p+4	?	A	A	$\alpha+4$
add	$\alpha+4$ 0/9/9/R/0/20	5020	10010000/10	010 +	A	A	A	A	4	0	?	4	$\alpha+8$
sw	$\alpha+8$ 8B/8/R/0	0	X1X00100/00	010 +	?	p	4	0	p	?	?	?	$\alpha+C$

Observatii:

- Valorile sunt scrise direct in hexa.
- A adaugat o coloana in plus **op/RS/RT/RD/0/FCT** (pentru instructiuni de tip R) si **op/RS/RT/imm** (pentru instructiuni de tip I). Fiecare variabila este scrisa in hexa, deci acel 23 de la lw inseamna $2*16+3=35$ (, care este fix opcode-ul de la lw: 100011)
- A adaugat o alta coloana: CTRL, care include toate variabilele din Unitatea de Control: RegDst, ALUSrc, MemToReg, RegWrite, MemRead, MemWrite, Branch, Jump, ALUOp.

LUMEA DACA FIECARE PROGRAMATOR CODA IN MIPS





References

- [1] Dumitru Daniel Drăgulici. *Curs Arhitectura Sistemelor de Calcul*.
- [2] Larisa Dumitrache. *Tutoriat 2019*
- [3] Bogdan Macovei. *Laboratoare ASC 2019/ 2020*